

Actividad Obligatoria I — Simulaciones HTTP y MQTT + captura con Wireshark

Licenciatura en Ciberdefensa — Dispositivos Remotos e Internet de las Cosas (FADENA / UNDEF) — 2026

Autor: Gustavo Courault — Fork: github.com/gcourault/iot_repo

1. Entorno y procedimiento

Ítem	Valor
Sistema	Debian 12, kernel 6.1
Wireshark / tshark / dumpcap	4.0.17
Python / gestor	3.12.14 / uv
Broker local	Mosquitto 2.0.11 (~/.mosquitto/mosquitto.conf, puertos 1883 y 8883/TLS)
Broker remoto	mqtt-dashboard.com y broker.hivemq.com (clúster público HiveMQ, AWS)
Interfaces capturadas	lo (loopback, tráfico local) y eno1 (Ethernet, IP 192.168.1.24/23, gateway 192.168.1.1)

Pasos realizados:

1. Fork de dracero/iot_repo y clonado (origin = fork propio, upstream = repo original).
2. uv sync para dependencias y ./setup_mosquitto_tls.sh para generar la CA, el certificado del servidor y la configuración de Mosquitto.
3. Por cada experimento se inició la captura con dumpcap **antes** de ejecutar los scripts, con filtro de captura por puerto:

Experimento	Scripts	Interfaz	Filtro	Archivo
HTTP	uvicorn lectura:app --port 8000 + sensor.py	lo	tcp port 8000	capturas/http_lo.pcapng
MQTT local	mosquitto + lectura_mq.py + sensor_mq.py	lo	tcp port 1883	capturas/mqtt_lo.pcapng
MQTT remoto	(mismos scripts, conexión a mqtt-dashboard.com)	eno1	tcp port 1883	capturas/mqtt_eno1.pcapng
MQTT + TLS local	sensor_er.py + lectura_er.py	lo	tcp port 8883	capturas/tls_lo.pcapng
MQTT + TLS remoto	(mismos scripts, conexión a broker.hivemq.com)	eno1	tcp port 8883	capturas/tls_eno1.pcapng

Los logs de cada script quedaron junto a las capturas en `capturas/*.log`. El análisis se hizo con Wireshark/tshark (Follow TCP Stream, Statistics → Conversations, disectores HTTP/MQTT/TLS).

2. HTTP (puerto 8000)

Captura: 96 paquetes en 10,9 s, 6 lecturas del sensor (una cada 2 s), todas respondidas 200 OK.

Capa 7 — Aplicación

Cada lectura es un `POST /telemetry HTTP/1.1` con cuerpo JSON y su respuesta JSON. `Follow TCP Stream` (stream 1) muestra **todo en texto plano**:

```
POST /telemetry HTTP/1.1
Host: localhost:8000
User-Agent: python-requests/2.32.5
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Length: 37
Content-Type: application/json

{"sensor_id": "S-001", "valor": 26.3}

HTTP/1.1 200 OK
date: Mon, 14 Sep 2026 14:50:29 GMT
server: uvicorn
content-length: 51
content-type: application/json

{"status": "recibido", "sensor": "S-001", "valor": 26.3}
```

- Se leen el método, la URI, las cabeceras y el cuerpo con el ID del sensor y el valor.
- Las cabeceras exponen además el software de ambos extremos (`python-requests/2.32.5`, `server: uvicorn`), información útil para un atacante (*fingerprinting*).
- Wireshark decodifica el JSON campo por campo (Member: `sensor_id` → `S-001`, Member: `valor` → `26.3`).
- Tiempo de respuesta del servidor (`http.time`): **1,2 a 2,3 ms**.

Capa 4 — Transporte (TCP)

- **Cliente → servidor**: puertos efímeros `53620`, `53630`, `53642`, `53656`, `49748`, `49756` hacia el puerto **8000**.
- **Una conexión TCP nueva por cada lectura** (`requests.post` no reutiliza la sesión). El ciclo completo del stream 1 es:

Trama	Flags	Contenido
3	SYN	53620 → 8000, MSS=65495, WS=128
4	SYN, ACK	8000 → 53620
5	ACK	fin del <i>3-way handshake</i>
6	PSH, ACK	cabeceras del POST (208 bytes)
8	PSH, ACK	cuerpo JSON (37 bytes)
10	PSH, ACK	cabeceras de la respuesta (125 bytes)
12	PSH, ACK	cuerpo JSON de la respuesta (51 bytes)
7, 9, 11, 13	ACK	acuses de recibo
27-29	FIN, ACK / FIN, ACK / ACK	cierre ordenado ≈2 s después

- Una lectura HTTP ocupa **14 tramas y 1.361 bytes** en IPv4, de los cuales solo 37 son el dato útil.
- **Hallazgo:** antes de cada conexión IPv4 el cliente intenta primero IPv6 (`::1` → 8000, stream 0, 2, 4...), porque `localhost` resuelve primero a `::1`. El servidor responde con **RST, ACK**, ya que uvicorn solo escucha en `127.0.0.1` (log: `Uvicorn running on http://127.0.0.1:8000`). El cliente cae entonces a IPv4. Son 12 conversaciones en total: 6 fallidas por IPv6 y 6 exitosas por IPv4.
- No hubo retransmisiones ni otras anomalías TCP (`tcp.analysis.flags = 0`).

Capa 3 — Red (IP)

- IPv4 `127.0.0.1` → `127.0.0.1` (y `::1` → `::1` en los intentos IPv6): **tráfico interno**, nunca sale de la máquina.
- TTL = 64, flag *Don't Fragment* activado, protocolo 6 (TCP).

Capa 2 — Enlace

- En `lo` Linux encapsula como Ethernet II con **MAC origen y destino** `00:00:00:00:00:00`: no hay direcciones físicas reales.
- EtherType `0x0800` (IPv4) o `0x86dd` (IPv6).

¿Legible?

Sí, completamente: cabeceras, URL y datos del sensor en claro. La cadena `S-001` aparece en crudo 12 veces en el `.pcapng`.

3. MQTT sin cifrar (puerto 1883)

Los scripts publican y se suscriben **en paralelo a dos brokers**: Mosquitto local y `mqtt-dashboard.com`, ambos con el tópico `fadena/test`. Hubo 7 publicaciones, recibidas por `lectura_mq.py` desde los dos brokers.

3.1 Broker local (`lo`) — 51 paquetes

Capa 7 — Aplicación (MQTT)

Secuencia observada:

Trama	Cliente	Mensaje MQTT	Detalle
4	lector	CONNECT	Client ID <code>lectura-mq-local</code>
6	broker	CONNACK	
8	lector	SUBSCRIBE (id=1)	tópico <code>fadena/test</code>
9	broker	SUBACK (id=1)	
14	sensor	CONNECT	Client ID <code>sensor-mq-local</code>
16	broker	CONNACK	
18	sensor → broker	PUBLISH	<code>fadena/test</code> , QoS 0
19	broker → lector	PUBLISH	reenvío al suscriptor 52 µs después
22, 24, ... 42, 44		PUBLISH	uno cada 2 s

- **QoS 0 (*at most once*)**: no hay PUBACK, solo el ACK de TCP. Un suscriptor que no está conectado pierde el mensaje.

- El broker desacopla emisor y receptor (**publicación/suscripción**): el sensor nunca abre una conexión hacia el lector.
- Disección del PUBLISH (trama 18):

```
Header Flags: 0x30 → Message Type: Publish (3), DUP=0, QoS=0, Retain=0
Msg Len: 94
Topic Length: 11
Topic: fadena/test
Message: 7b2273656e736f725f6964223a2022532d303031222c...
```

El volcado hexadecimal de la misma trama muestra el payload **legible en ASCII**:

```
0050  5c aa 30 76 5b c0 30 5e 00 0b 66 61 64 65 6e 61  \.0v[.0^..fadena
0060  2f 74 65 73 74 7b 22 73 65 6e 73 6f 72 5f 69 64  /test{"sensor_id
0070  22 3a 20 22 53 2d 30 30 31 22 2c 20 22 76 61 6c  ": "S-001", "val
0080  6f 72 22 3a 20 32 37 2e 33 31 2c 20 22 74 69 6d  or": 27.31, "tim
0090  65 73 74 61 6d 70 22 3a 20 22 32 30 32 36 2d 30  estamp": "2026-0
00a0  39 2d 31 34 54 31 31 3a 35 32 3a 30 32 2e 32 33  9-14T11:52:02.23
00b0  37 35 32 39 22 7d                                7529"}

```

30 5e es la cabecera fija (PUBLISH, longitud 94), 00 0b la longitud del tópic y a continuación vienen el tópic y el JSON. Wireshark muestra el campo *Message* en hexadecimal, pero **no está cifrado ni codificado**: son los bytes ASCII del JSON. Con la preferencia del protocolo MQTT *"Show Publish Message as text"* se ve directamente como texto.

Capa 4 — Transporte

- Dos conexiones TCP **persistentes**: lector ::1:59527 ↔ ::1:1883 y sensor ::1:40385 ↔ ::1:1883.
- Handshake SYN / SYN-ACK / ACK una sola vez; después cada lectura viaja en **un único segmento PSH,ACK**.
- MSS = 65476, porque la MTU de loopback es de 65536.
- Un PUBLISH ocupa **182 bytes en total**: 14 de Ethernet, 40 de IPv6, 32 de TCP y 96 de MQTT (81 son el JSON). Una lectura HTTP ocupa 1.361 bytes.
- Cierre: FIN,ACK / FIN,ACK / ACK (tramas 46–51). **No hay mensaje MQTT DISCONNECT**, porque los scripts se interrumpieron con Ctrl+C y `loop_stop()` quedó bloqueado. Mosquitto lo registra como `Client sensor-mq-local closed its connection`.
- No hubo retransmisiones.

Capa 3 — Red

- **IPv6 ::1 → ::1**: Mosquitto escucha en IPv4 e IPv6 y `localhost` resuelve primero a ::1. Es tráfico interno.
- Hop Limit = 64.

Capa 2 — Enlace

- MAC 00:00:00:00:00:00 en ambos extremos (loopback), EtherType 0x86dd.

3.2 Broker remoto `mqtt-dashboard.com` (eno1) — 51 paquetes

Capa 7

- Misma secuencia CONNECT → CONNACK → SUBSCRIBE → SUBACK → PUBLISH, con Client IDs `lectura-mq-remote` y `sensor-mq-remote`.
- Tópico y payload **igual de legibles**, pero esta vez **viajando por Internet**.
- Detalle: en la trama 16 el sensor envía el PUBLISH **antes** de recibir el CONNACK (trama 17). La librería paho encola la publicación sin esperar la confirmación.

Capa 4

Cliente	Origen	Destino
lector	192.168.1.24:55319	18.192.161.231:1883
sensor	192.168.1.24:44563	18.195.141.56:1883

- `mqtt-dashboard.com` resuelve a 3 IP de AWS (54.93.69.189, 18.195.141.56, 18.192.161.231). Sensor y lector cayeron en **nodos distintos del clúster** y el mensaje igual se entregó.
- MSS = 1460 (MTU Ethernet de 1500).
- **RTT del handshake** (SYN → SYN/ACK): **232–239 ms**.
- **Latencia de extremo a extremo** (PUBLISH del sensor → PUBLISH entregado al lector): 4,8767 s → 5,1118 s ≈ **235 ms**, contra **52 µs** en el broker local.
- Cada PUBLISH ocupa 162 bytes (14 Ethernet + 20 IPv4 + 32 TCP + 96 MQTT). No hubo retransmisiones.

Capa 3

- IPv4 192.168.1.24 (privada, detrás de NAT) ↔ IP públicas de AWS: **tráfico externo**. Se usa IPv4 porque eno1 no tiene IPv6 global.
- **TTL**: salida = 64; llegada = **246**. Si el servidor parte de 255, hay unos **9 saltos** de distancia.

Capa 2

Dirección	MAC origen	MAC destino
saliente	f4:b5:20:4e:f1:9b (Biostar — NIC de esta PC)	d0:07:ca:75:69:f0 (Juniper — router/gateway 192.168.1.1)
entrante	d0:07:ca:75:69:f0	f4:b5:20:4e:f1:9b

- Aquí **sí hay MAC reales**. La MAC destino **no es la del broker sino la del siguiente salto** (el gateway): la capa 2 solo tiene alcance dentro de la LAN.
- En la red local sirve para identificar qué equipo generó el tráfico (OUI del fabricante).

¿Legible?

Sí. Client IDs, tópico y JSON en claro, tanto en local como en el tráfico que sale a Internet (S-001 aparece 14 veces en crudo en cada captura). Además los brokers aceptan conexiones anónimas, así que cualquiera en el camino (o cualquier cliente suscripto a `fadena/test`) puede **leer e incluso inyectar** lecturas falsas.

4. MQTT sobre TLS (puerto 8883) — `sensor_er.py` / `lectura_er.py`

Hubo 5 publicaciones exitosas en ambos brokers y un *error de red simulado* con reintento. Ese error es una excepción generada por el script antes de publicar, así que **no deja rastro en la red**: solo aparece un hueco temporal, sin retransmisiones TCP. El lector, que arrancó 6 s después, perdió la primera lectura (QoS 0, sin *retain*).

4.1 Broker local (`lo`) — 61 paquetes

- **Stream 0** (`127.0.0.1:34350 → 8883`):
 - Secuencia SYN, SYN/ACK, ACK, FIN. Es la verificación `_broker_running()` de `sensor_er.py`, que abre y cierra el socket sin hablar TLS.
 - Mosquitto contesta con **TLS Alert (Fatal, Decode Error)** y el cliente envía **RST**.
- **Conexiones reales**: `::1:46981 ↔ ::1:8883` y `::1:48069 ↔ ::1:8883` (IPv6 loopback, MAC en cero).
- **Handshake**:
 - **Client Hello** (en claro): **SNI** = `localhost`, versiones soportadas TLS 1.3 / 1.2 y 17 suites de cifrado ofrecidas.
 - **Server Hello**: **TLS 1.3** con `TLS_AES_256_GCM_SHA384` (0x1302).
 - En TLS 1.3 el certificado del servidor ya viaja cifrado, así que ni siquiera se ve.
- **Datos**: después del handshake todo son registros **Application Data** opacos. Un fragmento del volcado hexadecimal:

```
0050  62 fa 30 77 55 b9 17 03 03 00 78 b1 53 bd f9 0f   b.0wU.....x.S...
0060  84 9f 0a a4 f9 e3 c8 3d 72 8c f5 93 6c 20 29 59   .....=r...l )Y
0070  14 27 30 3a ad 9e d5 ca 1e 96 5c a5 d3 94 06 13   .'0:.....\.....
```

`17 03 03 00 78` es la cabecera del registro TLS (tipo 23 = *application data*, 120 bytes); el resto es texto cifrado.

- **Wireshark no decodifica ningún paquete MQTT** (filtro `mqtt → 0` resultados).
- Buscando `SENSOR_ER` o `fadena/test` en los bytes crudos del `.pcapng`: **0 coincidencias**.

4.2 Broker remoto `broker.hivemq.com` (`eno1`) — 56 paquetes

- **Conexiones**: `192.168.1.24:44401 ↔ 54.93.69.189:8883` y `192.168.1.24:49861 ↔ 18.195.141.56:8883`. Son las mismas IP que `mqtt-dashboard.com`: el mismo clúster HiveMQ.
- **MAC y TTL**: Biostar → Juniper, TTL 64 de salida y 246 de llegada, igual que en 3.2.
- **Client Hello**: **SNI** = `broker.hivemq.com` visible en claro, ofrece TLS 1.3 y 1.2.
- **Server Hello**: el servidor elige **TLS 1.2** con `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256` (0xc02f).
- **Certificado**: como es TLS 1.2, el mensaje *Certificate* **viaja en claro**. Wireshark muestra la cadena:
 - Titular: `CN=mqttdashboard.com`
 - Emisor: `Amazon RSA 2048 M04`
 - Raíces: `Amazon Root CA 1` y `Starfield Services Root CA – G2`
 - El cliente lo validó con los certificados del sistema.
- **Secuencia**: Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message y a partir de ahí solo Application Data.
- **Resultado**: 0 paquetes MQTT disecados y 0 apariciones de `SENSOR_ER` o `fadena/test` en crudo. No hubo retransmisiones.

¿Legible?

No. Tópico, Client ID y payload están cifrados. Sin las claves de sesión Wireshark no puede mostrarlos (se podrían descifrar exportando `SSLKEYLOGFILE` desde el cliente).

Lo que **sí sigue expuesto** (metadatos):

- IP y puertos, MAC en la LAN.
- SNI con el nombre del broker.
- En TLS 1.2, el certificado del servidor.
- Tamaño y periodicidad de los registros: `Application Data` de ~127–266 bytes cada 5 s, suficiente para inferir que un sensor publica cada 5 s (**análisis de tráfico**).

5. Comparación

Aspecto	HTTP	MQTT (1883)	MQTT + TLS (8883)
Transporte	TCP	TCP	TCP + TLS
Puerto servidor	8000	1883	8883
Modelo	petición/respuesta, cliente → servidor	publicación/suscripción vía broker	publicación/suscripción vía broker
Conexión	nueva por cada lectura (handshake cada 2 s)	persistente	persistente (handshake TCP + TLS una vez)
Tramas/bytes por lectura	14 tramas, 1.361 bytes	1 trama, 182 bytes (lo) / 162 (eth)	1 registro cifrado ~127–266 bytes
Confirmación de entrega	respuesta 200 OK	QoS 0: ninguna (solo ACK TCP)	QoS 0: ninguna
Payload visible	Sí (cabeceras + JSON)	Sí (tópico + JSON + Client ID)	No
Metadatos visibles	todo	todo	IP, puertos, SNI, tamaños, tiempos (+ certificado en TLS 1.2)
Local (lo)	IPv4 127.0.0.1 (IPv6 rechazado con RST)	IPv6 ::1	IPv6 ::1, TLS 1.3
Remoto (eno1)	—	IPv4, RTT ≈ 235 ms, TTL 246	IPv4, TLS 1.2
Capa 2	MAC 00:00:.... (loopback)	loopback: 00:00:....; remoto: PC (Biostar) → gateway (Juniper)	idem

6. Conclusiones

- **Protocolos capturados:** HTTP/1.1 sobre TCP/8000, MQTT 3.1.1 sobre TCP/1883 (broker local y público) y MQTT sobre TLS en TCP/8883 (broker local con TLS 1.3 y broker público con TLS 1.2).
- **¿Claro o cifrado?** HTTP y MQTT/1883 van **en texto plano**: método, URL, cabeceras, Client ID, tópico y valores del sensor son legibles directamente en Wireshark, incluso cuando el tráfico sale a Internet. Con TLS/8883 el contenido es **ilegible** (solo `Application Data`), aunque siguen expuestos metadatos como IP, puertos, SNI, tamaños y tiempos.
- **Qué se vio en cada capa:**
 - **Capa 7:** mensajes de aplicación (POST/200 OK; CONNECT/CONNACK/SUBSCRIBE/SUBACK/PUBLISH; handshake TLS).

- **Capa 4:** puertos efímeros del cliente hacia 8000/1883/8883, *3-way handshake*, PSH/ACK con datos, cierre FIN, RST ante un puerto sin servicio (IPv6 en HTTP), sin retransmisiones.
- **Capa 3:** 127.0.0.1/::1 para el tráfico interno y 192.168.1.24 ↔ IP públicas de AWS para el externo; TTL 64 de salida y 246 de llegada (~9 saltos).
- **Capa 2:** MAC nulas en loopback; en `eno1`, MAC de la PC → MAC del gateway (no la del servidor).
- **Eficiencia:** MQTT es mucho más liviano que HTTP para telemetría: 1 trama de 162–182 bytes contra 14 tramas y 1.361 bytes por lectura, gracias a la conexión persistente y a una cabecera fija de 2 bytes. Por eso se prefiere en IoT.
- **Visión de ciberdefensa:** MQTT en 1883 contra un broker público y anónimo expone los datos del sensor a cualquiera en el camino y permite suscribirse o inyectar mensajes falsos en el tópico. Para uso real hace falta **TLS (8883) + autenticación (usuario/contraseña o certificado de cliente) + ACL por tópico**, y tener en cuenta que TLS no oculta los patrones de tráfico.

Anexo — Filtros usados y tramas sugeridas para capturas de pantalla

Captura	Filtro de visualización	Qué mostrar
<code>http_lo.pcapng</code>	<code>tcp.port == 8000</code>	trama 8 (POST, capas expandidas); Follow → TCP Stream en stream 1; tramas 1–2 (IPv6 con RST)
<code>mqtt_lo.pcapng</code>	<code>mqtt</code>	tramas 4, 8, 18 (CONNECT/SUBSCRIBE/PUBLISH); bytes hex de la 18 con el JSON
<code>mqtt_eno1.pcapng</code>	<code>mqtt / tcp.flags.syn==1</code>	trama 16 con Ethernet expandido (MAC Biostar → Juniper), TTL 246 en respuestas
<code>tls_lo.pcapng</code>	<code>tls</code>	tramas 10 y 12 (Client/Server Hello TLS 1.3); Application Data; filtro mqtt vacío
<code>tls_eno1.pcapng</code>	<code>tls.handshake</code>	trama 4 (SNI), trama 8 (certificado Amazon en claro), Application Data

Comandos tshark útiles para reproducir el análisis:

```
tshark -r capturas/http_lo.pcapng -q -z follow,tcp,ascii,1
tshark -r capturas/mqtt_lo.pcapng -Y mqtt -T fields -e frame.number -e mqtt.msgtype -e mqtt.clientid -e mqtt.topic -e mqtt.msg
tshark -r capturas/tls_eno1.pcapng -Y tls.handshake -T fields -e frame.number -e tls.handshake.extensions_server_name -e tls.handshake.ciphersuite -e _ws.col.Info
tshark -r capturas/mqtt_eno1.pcapng -q -z conv,tcp
```